



中山大學
SUN YAT-SEN UNIVERSITY

信息安全技术期末考查

Watermark for LLM 三种算法对比实验

姓 名 _____ 待填写

学 号 _____ 待填写

学 院 _____ 待填写

专 业 _____ 待填写

2026 年 7 月 7 日

摘要

大语言模型生成文本的自动化能力正在快速提升，随之而来的学术诚信、虚假信息生成和合成数据污染等问题，使文本水印成为信息安全领域的重要研究方向。本文围绕三种代表性 LLM 水印方案展开复现与对比：KGW 通过伪随机绿名单和 logit 偏置在生成阶段嵌入统计信号；SWEET 针对代码等低熵场景，只在高熵 token 上嵌入和检测水印；EWD 则在检测阶段引入熵加权统计量，为高熵 token 分配更高权重。实验实现了从数据读取、模型生成、水印嵌入、水印检测、perplexity 评估到结果作图的完整链路。正式云端实验使用 OPT-1.3B 生成、OPT-6.7B 计算 PPL，在 C4 RealNewsLike 500 条新闻文本和 200 条低熵结构化代码文本上评估三种方法。结果表明，三种方法均能有效区分 human 与 watermarked 文本；SWEET 在保持高检测性能的同时带来更低 PPL 增量；EWD 适合作为 KGW 生成文本的增强检测器，但本身不是独立生成水印方案。

1 背景和动机

生成式大语言模型已经能够完成新闻写作、课程作业、代码生成和问答等任务。模型输出越接近人类文本，越难通过人工方式判断来源，这会带来三个典型风险。第一，恶意用户可以批量生成虚假新闻、评论或社交媒体内容，增加内容审核难度。第二，学生或程序员可能提交模型生成文本或代码，造成学术诚信与版权归属争议。第三，如果未来训练数据中混入大量机器生成内容，模型训练和评测本身也可能被污染。因此，能够在文本生成阶段主动嵌入、在事后自动检测的水印机制，是一类具有现实意义的信息安全技术。

传统数字水印常见于图像、音频和视频，其核心是在人类难以察觉的载体空间中嵌入可检测信号。LLM 文本水印的困难在于文本是离散 token 序列，任何 token 修改都可能改变语义、语法或代码功能。KGW [1] 提出了一个简单有效的方向：不直接修改已生成文本，而是在模型采样时轻微提高部分 token 的概率，让最终文本在统计上包含更多“绿名单 token”。SWEET [2] 观察到代码生成等任务的 token 熵较低，强行偏置低熵 token 会破坏功能，偏置太弱又难以检测，于是提出只处理高熵 token。EWD [3] 则认为检测阶段也应充分利用熵信息，不能把所有 token 一视同仁。三者形成从“基础生成水印”到“选择性生成水印”再到“熵加权检测”的发展脉络。

2 方法原理

2.1 KGW 水印

KGW 的基本思想是：给定前文 token 和密钥，通过哈希函数确定一个伪随机种子，再把词表划分为绿名单 G 和红名单 R 。绿名单比例记为 γ 。在生成第 t 个 token 时，模型先输出原始 logit 向量，然后对所有绿名单 token 的 logit 加上偏置 δ ，再进行采样。

若原始分布较高熵，绿名单 token 的采样概率会明显高于 γ ；若文本不是由该水印生成，则 token 落入绿名单的概率近似为 γ 。

检测时，检测器根据同一密钥和前文 token 复原每个位置的绿名单，统计待检测文本中绿名单 token 数 $|s|_G$ 。对于长度为 T 的文本，KGW 使用一侧 z 检验：

$$z = \frac{|s|_G - \gamma T}{\sqrt{T\gamma(1-\gamma)}}.$$

当 z 超过阈值时，判定文本含有水印。论文中常用 $z > 4$ 作为较严格阈值，对应较低误报概率。

2.2 SWEET 水印

KGW 在自然语言新闻生成等高熵场景中效果较好，但在代码生成中会遇到两类问题：低熵 token 往往由语法或上下文强约束决定，强行提高绿名单概率可能破坏代码正确性；同时，低熵 token 很难被水印偏置改变，继续把它们计入检测统计量会稀释 z -score。SWEET 因此引入熵阈值 τ 。在生成阶段，只有当当前位置模型分布熵 $H_t > \tau$ 时才执行 KGW 式绿名单 logit 偏置；在检测阶段，也只统计熵高于阈值的 token。

设高熵 token 数为 N^h ，其中绿名单 token 数为 N_G^h ，SWEET 的检测统计量为：

$$z = \frac{N_G^h - \gamma N^h}{\sqrt{N^h \gamma (1 - \gamma)}}.$$

这相当于过滤掉难以被水印影响的低熵位置，使检测更集中于水印真正可能改变采样结果的区域。

2.3 EWD 检测

EWD 的核心贡献不在于提出新的生成器，而在于改造检测统计量。KGW 给所有 token 相同权重，SWEET 给高熵 token 权重 1、低熵 token 权重 0。EWD 认为 token 对检测结论的影响应与熵连续相关：高熵 token 更容易被水印偏置改变，因此应在检测中占更高权重；低熵 token 即使不在绿名单，也不应强烈削弱水印判断。

本文实现采用 EWD 论文中的线性权重形式。对第 i 个被检测 token，根据模型分布计算 spike entropy SE_i ，再令权重 $W_i = \max(SE_i - C_0, 0)$ 。检测统计量为：

$$z' = \frac{|s|_G - \gamma \sum_i W_i}{\sqrt{\gamma(1-\gamma) \sum_i W_i^2}},$$

其中 $|s|_G$ 表示绿名单 token 权重和。EWD 可以用于 KGW 或 SWEET 生成的文本；本实验为了隔离变量，默认让 EWD 复用 KGW 生成文本，只替换检测器。

3 实验设计

3.1 模型和数据

正式实验遵循统一模型原则。生成模型配置为 facebook/opt-1.3b, PPL/oracle 模型配置为 facebook/opt-6.7b, OPT 模型来自 Open Pre-trained Transformer 系列 [4]。检测和生成均使用同一密钥、同一绿名单比例 $\gamma = 0.25$ 、同一 logit 偏置 $\delta = 2.0$ 。

主实验使用 C4 的 RealNewsLike 子集 [5], 共 500 条。每条样本文本被切分为 prompt 和 human completion, prompt 输入生成模型, 生成长度设为 200 tokens; 同一条 prompt 分别生成无水印文本、KGW 水印文本和 SWEET 水印文本, EWD 对 KGW 水印文本进行加权检测。

补充实验使用 200 条低熵 Python 结构化代码样本。该数据集由项目脚本生成, 每条样本包含固定风格的函数签名、任务说明和短函数体, 用于模拟低熵、强结构 token 场景。需要说明的是, 它是低熵压力测试数据, 不等同于 HumanEval 或 MBPP 的真实代码生成基准。

3.2 评价指标

本文使用以下指标对比三种算法:

- z-score: 检测统计量, 越高表示越可能含有水印。
- AUROC: 将 human completion 作为负样本、水印生成文本作为正样本, 衡量检测分数排序能力。
- TPR@5%FPR: 在误报率不超过 5% 时的检测召回率。
- $z > 4$ 检出率和误报率: 对应 KGW 论文中常用的严格检测阈值。
- PPL: 用 OPT-6.7B oracle 模型计算生成 completion 的困惑度, 反映生成质量变化。
- 可计分 token 数和绿名单比例: 用于解释水印嵌入强度和检测分数。

3.3 代码实现

项目代码实现了完整实验链路。水印模块包含 KGW 和 SWEET 的生成处理器, 可直接接入 HuggingFace 生成流程; 同一模块还实现 KGW、SWEET 和 EWD 三类检测器。为保证检测和生成跨 CPU/GPU 一致, 绿名单伪随机排列固定在 CPU 上生成, 再搬运到目标设备。实验入口负责读取配置、加载模型、生成文本、计算检测分数和 PPL, 并输出 CSV/JSONL; 作图脚本根据结果绘制 z-score 分布、ROC 曲线和质量-可检测性权衡图。

4 实验结果与分析

4.1 C4 RealNewsLike 主实验

表 1 给出 C4 RealNewsLike 500 条主实验结果。三种方法都能有效区分 human 与 watermarked 文本，AUROC 均高于 0.995。在 $z > 4$ 的固定阈值下，三种方法的误报率均为 0，检出率分别为 92.2%、94.6% 和 94.0%。

表 1: C4 RealNewsLike 500 条主实验结果

算法	水印 z	human z	AUROC	TPR@5%FPR	$z > 4$ 检出	$z > 4$ 误报	平均 PPL	PPL 增量
KGW	8.839	-0.134	0.9955	0.990	92.2%	0%	21.388	+7.986
SWEET	9.716	-0.057	0.9982	0.994	94.6%	0%	17.239	+3.837
EWD	9.669	-0.056	0.9985	0.988	94.0%	0%	21.388	+7.986

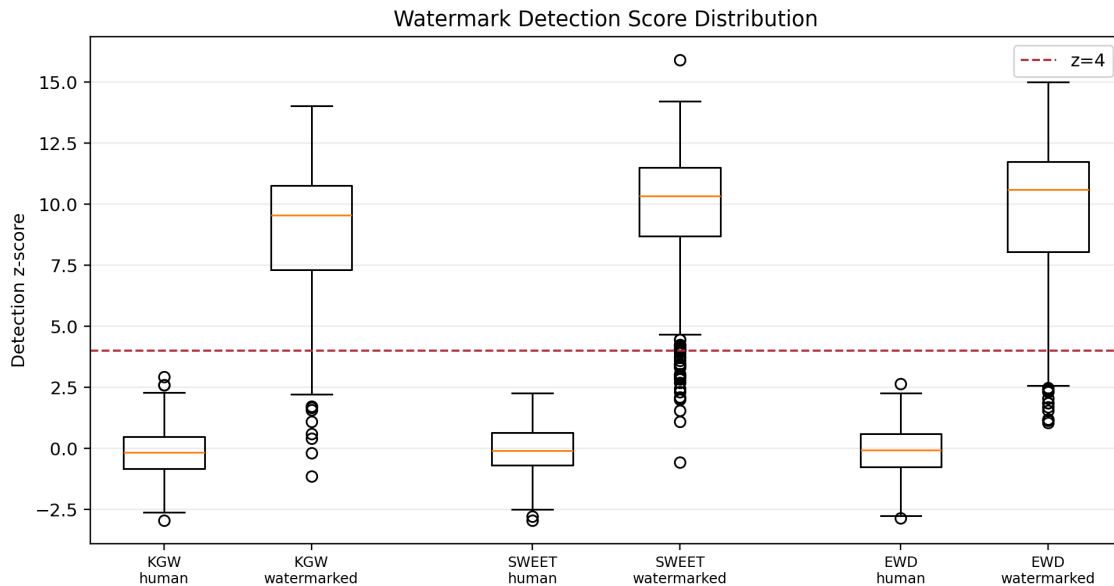


图 1: C4 主实验中 human 与 watermarked z-score 分布

图 1 显示，human completion 的 z-score 集中在 0 附近，而水印文本整体明显高于 $z = 4$ 阈值。KGW 和 SWEET 均存在少量低 z-score 离群点，主要来自过短生成或可计分 token 较少的样本；这说明固定阈值检测仍受文本长度影响。SWEET 的中位数 z-score 更高，同时 PPL 增量更低，说明选择性跳过低熵位置在高熵新闻场景中也能降低质量损失。

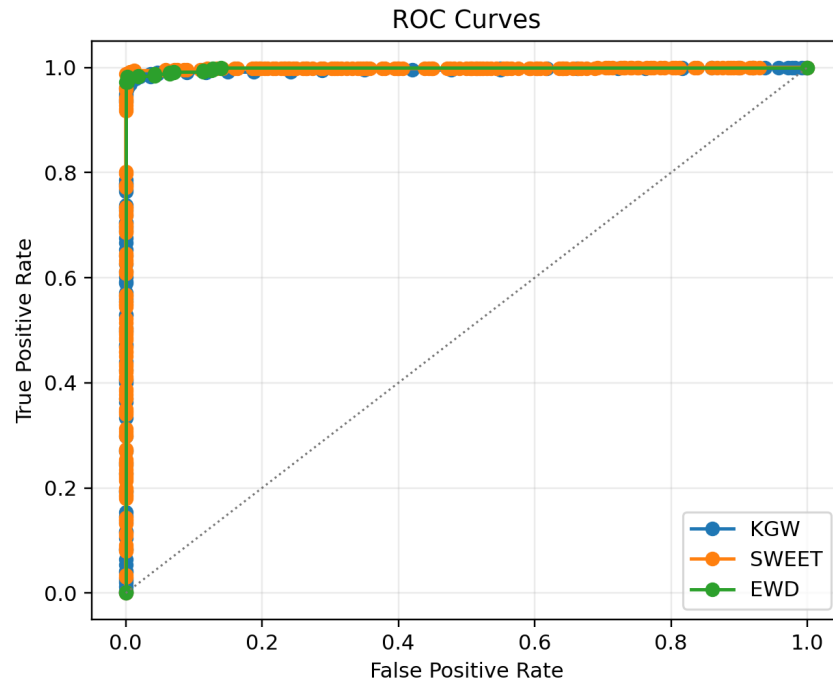


图 2: C4 主实验 ROC 曲线

图 2 中三条 ROC 曲线均贴近左上角，说明排序意义上的检测能力很强。EWD 的 AUROC 最高，但 EWD 复用 KGW 生成文本，因此不能把它理解为独立生成算法优于 KGW；更准确的表述是：在 KGW 生成文本上，EWD 熵加权检测能略微改善检测分数排序。

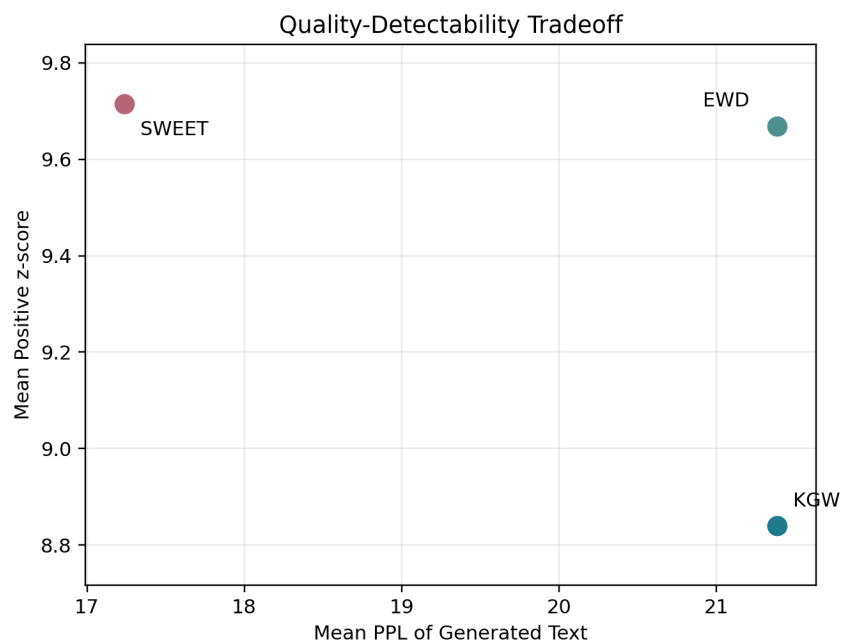


图 3: C4 主实验 PPL 与检测 z-score 权衡

图 3 展示了质量与可检测性的权衡。SWEET 位于更低 PPL、更高 z-score 的区域，说明它在本实验中取得了更好的质量-检测折中。KGW 和 EWD 的 PPL 相同，是因为 EWD 直接复用 KGW 的生成文本。C4 500 中存在一个极端样本只生成了极短 completion，导致 KGW/EWD PPL 达到 1385.43，并拉高平均 PPL；剔除 PPL 大于 100 的极端值后，KGW 平均 PPL 约为 18.655，SWEET 约为 16.965，SWEET 仍然更优。

4.2 低熵代码补充实验

表 2 给出低熵代码补充实验结果。三种方法的 AUROC 和 TPR@5%FPR 均达到 1.0，说明在该合成结构化数据上 human 与 watermarked 的分离非常明显。

表 2: 低熵代码 200 条补充实验结果

算法	水印 z	human z	AUROC	TPR@5%FPR	$z > 4$ 检出	$z > 4$ 误报	平均 PPL	PPL 增量
KGW	9.110	-2.252	1.000	1.000	97.0%	0%	16.060	+1.569
SWEET	9.013	-0.867	1.000	1.000	98.0%	0%	15.570	+1.078
EWD	8.932	-1.082	1.000	1.000	98.0%	0%	16.060	+1.569

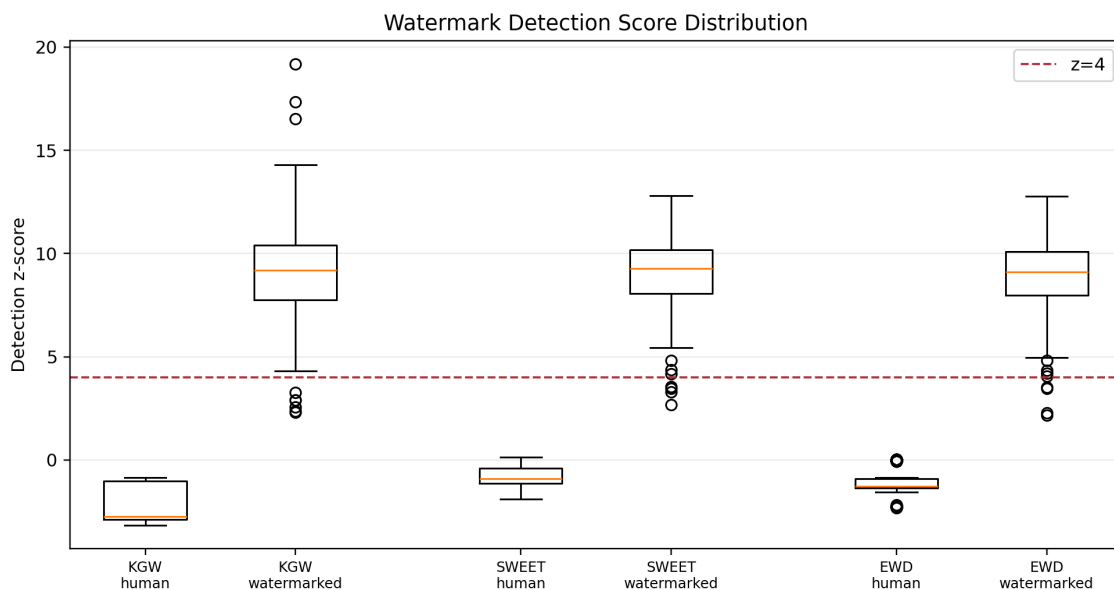


图 4: 低熵代码补充实验中 human 与 watermarked z-score 分布

图 4 显示，低熵代码数据中的 human z-score 明显偏负，watermarked z-score 则集中在较高区域，因此 ROC 指标达到饱和。SWEET 平均统计 87.18 个 token，而 KGW/EWD 平均统计 121.20 个 token，说明 SWEET 确实跳过了更多低熵位置；同时 SWEET 的 PPL 增量最低，符合其减少低熵 token 干预的设计目标。

4.3 方法优缺点对比

KGW 的优点是结构简单、检测不需要访问模型参数，只要知道密钥和哈希规则即可复原绿名单，因此部署成本低、统计解释清晰。缺点是低熵文本中水印信号可能被固定语法或上下文约束削弱，若提高 δ 又可能损害文本质量。

SWEET 解决的是“哪些 token 值得水印化”的问题：它跳过低熵位置，通常能降低对代码正确性或固定表达的破坏，同时提高有效统计样本中的绿名单比例。当前两组实验中，SWEET 都取得了最低 PPL 增量，说明它在质量保持方面有优势。

EWD 解决的是“检测时怎样使用 token”的问题：它不再二值化地保留或删除 token，而是根据熵连续加权，因此比 KGW 更重视高熵 token，也比 SWEET 更少依赖人工阈值。不过 EWD 仍需要模型 logits，且本身不是独立生成水印方案。

5 运行效果与复现

主实验命令如下：

```
python scripts/prepare_assets.py --config configs/opt_full.yaml --hf-mirror --
skip-models --dataset-samples 500 --dataset-output data/c4_realnewslike_500
.jsonl
python scripts/run_experiment.py --config configs/opt_cloud_c4_500_ppl.yaml --
output-dir results/opt_cloud_c4_500
python scripts/plot_results.py --results-dir results/opt_cloud_c4_500
```

低熵代码补充实验命令如下：

```
python scripts/make_low_entropy_code_dataset.py --output data/
code_low_entropy_200.jsonl --samples 200
python scripts/run_experiment.py --config configs/
opt_cloud_code_low_entropy_ppl.yaml --output-dir results/
opt_cloud_code_low_entropy
python scripts/plot_results.py --results-dir results/opt_cloud_code_low_entropy
```

当前实现会同时加载 OPT-1.3B 生成模型和 OPT-6.7B PPL/oracle 模型。若云端显存不足，`device_map: auto` 会触发 CPU offload，运行速度会明显下降。后续可改造为两阶段流程：先只加载 OPT-1.3B 完成生成和检测，再释放显存并加载 OPT-6.7B 补算 PPL。

6 总结

本文完成了 KGW、SWEET、EWD 三种 LLM 水印算法的复现式对比。三者的关系可以概括为：KGW 提供基础绿名单偏置框架，SWEET 在生成和检测阶段引入熵阈

值选择，EWD 在检测阶段引入连续熵权重。云端正式实验使用 OPT-1.3B/OPT-6.7B，在 C4 RealNewsLike 500 条主实验和 200 条低熵代码补充实验上均验证了水印检测的有效性。

实验结果表明，在高熵新闻文本中，三种方法均能稳定区分 human 与 watermarked 文本，且 $z > 4$ 阈值下没有误报；在低熵结构化代码中，SWEET 通过跳过低熵 token 获得更低 PPL 增量，体现出其质量保持优势。EWD 在本项目中应表述为“KGW 生成 + EWD 熵加权检测”，不能与 KGW、SWEET 完全并列为独立生成算法。

从安全角度看，文本水印的价值不在于给出绝对不可绕过的证明，而在于提供低成本、可量化、可审计的来源证据。未来可以进一步扩展 HumanEval/MBPP 等真实代码数据集，并增加同义改写、回译、截断、拼接攻击等鲁棒性实验。

小组贡献

姓名与贡献待填写。建议提交前补充每位组员在论文阅读、算法实现、实验运行、报告撰写和视频录制中的具体工作。

References

- [1] John Kirchenbauer et al. “A Watermark for Large Language Models”. In: *Proceedings of the 40th International Conference on Machine Learning*. PMLR. 2023, pp. 17061–17084.
- [2] Taehyun Lee et al. “Who Wrote this Code? Watermarking for Code Generation”. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. 2024, pp. 4890–4911.
- [3] Yijian Lu et al. “An Entropy-based Text Watermarking Detection Method”. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. 2024, pp. 11724–11735.
- [4] Susan Zhang et al. “OPT: Open Pre-trained Transformer Language Models”. In: *arXiv preprint arXiv:2205.01068* (2022).
- [5] Colin Raffel et al. “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer”. In: *Journal of Machine Learning Research* 21.140 (2020), pp. 1–67.